STROJNO UČENJE
12. predavanje

Kombiniranje modela

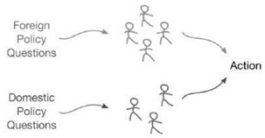
AI Center Lipik

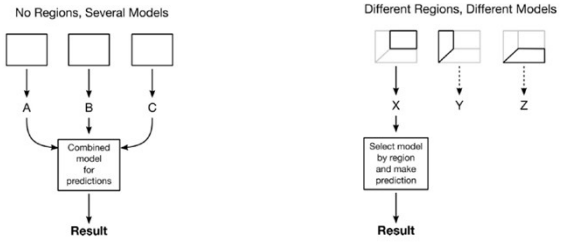
THE FUTURE IS HERE

12.1. Ansambli (skupovi) modela

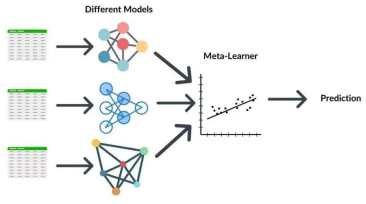
- IDEJA: klape se oslanjaju na različite glasove kako bi stvorili glazbenu ljepotu, različiti sustavi strojnog učenja mogu se kombinirati kako bi se poboljšale njihove pojedinačne komponente.

Primjer: ministarstvo unitarnjih i vanjskih poslova VS „ministarstvo poslova”





12.2. Glasački ansambli (stacking)



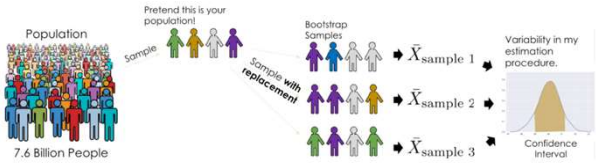
Google colab

```
from sklearn import datasets, model_selection as skms
from sklearn import linear_model, tree, naive_bayes, ensemble # Added ensemble
from sklearn import neighbors, metrics
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

digits = datasets.load_digits()
digits_ftrs, digits_tgt = digits.data, digits.target

diabetes = datasets.load_diabetes()
diabetes_ftrs, diabetes_tgt = diabetes.data, diabetes.target

iris = datasets.load_iris()
tts = skms.train_test_split(iris.data, iris.target,
                            test_size=.75, stratify=iris.target)
(iris_train_ftrs, iris_test_ftrs,
 iris_train_tgt,  iris_test_tgt) = tts

base_estimators = [('LogisticRegression', linear_model.LogisticRegression(max_iter=1000)),
                   ('DecisionTreeClassifier', tree.DecisionTreeClassifier(max_depth=3)),
                   ('GaussianNB', naive_bayes.GaussianNB())]

ensemble_model = ensemble.VotingClassifier(estimators=base_estimators)
skms.cross_val_score(ensemble_model, digits_ftrs, digits_tgt)
ensemble_model.fit(digits_ftrs, digits_tgt)
```

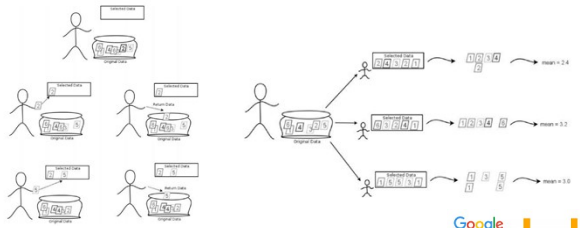
12.3. Bagging i slučajne šume

- Bagging – kratica od Bootstrap Aggregating - agregiranje putem bootstrap uzoraka
- Bootstrapping - ponovno uzorkovanje s povratom

Bootstraping ?



Uzorkovanje bez vraćanja i <u>sa vraćanjem</u>



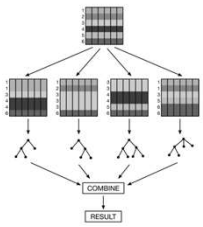
Google colab

```
import numpy as np
dataset = np.array([1,5,10,10,17,20,35])
def compute_mean(data):
    return np.sum(data) / data.size
compute_mean(dataset)

np.float64(14.0)

def bootstrap_sample(data):
    N   = len(data)
    idx = np.arange(N)
    bs_idx = np.random.choice(idx, N,
                              replace=True)
    return data[bs_idx]
```

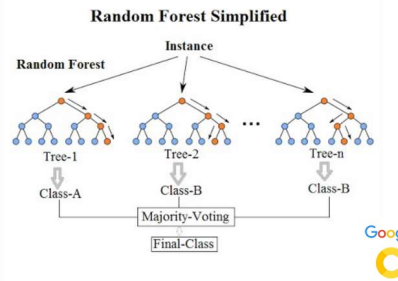
```
bsms = []
for i in range(5):
    bs_sample = bootstrap_sample(dataset)
    bs_mean = compute_mean(bs_sample)
    bsms.append(bs_mean)

    print(bs_sample, "{:5.2f}".format(bs_mean))
print("{:5.2f}".format(sum(bsms) / len(bsms)))

[ 1  1  1 10 20 10  5]  6.86
[10  5 17 35 35 20 10] 18.86
[ 5 10  5  1 10 17  5]  7.57
[20 20 10 10 17 10 17] 14.86
[10 35 10 10 10 10 10] 13.57
12.34
```

- Baggging proces se sastoji od slijedećih koraka:

1. Uzmite uzorak podataka sa vraćanjem
2. Napravite model i obučite ga na tim podacima.
3. Ponovimo



Random forest – slučajne šume

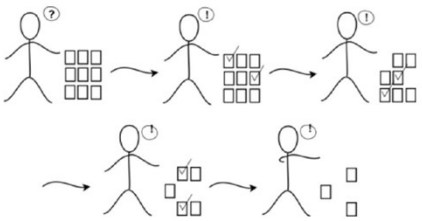


Google colab

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Učitavanje podataka
digits = load_digits()
X, y = digits.data, digits.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Random Forest s 100 stabala
clf = RandomForestClassifier(n_estimators=100, random_state=42, max_depth=None)
clf.fit(X_train, y_train)

# Evaluacija
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

# 1) Podaci
X, y = load_diabetes(return_X_y=True)

# 2) Train/Test podjela
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3) Model: slučajna šuma s 200 stabala
rf = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)

# 4) Treniranje i evaluacija
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
print("R² na testu:", r2_score(y_test, y_pred))
```

12.4. Boosting



- **Osnovne značajke boostinga**

1. **Slabi modeli** – obično stabla male dubine, brza i jednostavna.
2. **Sekvencijalno učenje** – modeli se treniraju jedan za drugim, a svaki novi model fokusira se na primjere koje su prethodni pogrešno klasificirali / loše predvidjeli.
3. **Težine uzoraka** – uzorcima se dodjeljuju težine; pogrešno klasificirani primjeri dobivaju veće težine u idućoj iteraciji.
4. **Agregacija** – konačna predikcija dobiva se kombinacijom svih slabih modela (npr. ponderirano glasanje ili zbrajanje predikcija).

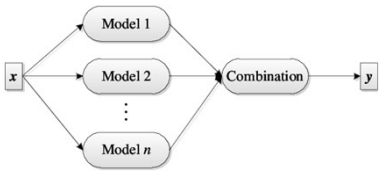
- **AdaBoost (Adaptive Boosting)**
  - Pionirski boosting algoritam.
  - Svakom uzorku daje početnu težinu. Nakon svakog modela, povećava težine za pogrešno klasificirane uzorke.
  - Konačna predikcija: ponderirano glasanje svih modela.
- **Gradient Boosting**
  - Svako novo stablo trenira se da predvidi rezidual (pogrešku) prethodnih modela.
  - Vrlo fleksibilan, može koristiti različite funkcije gubitka.

Google colab

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.ensemble import GradientBoostingClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
import numpy as np

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

gb = GradientBoostingClassifier(
    n_estimators=150, learning_rate=0.1, max_depth=2, random_state=42)

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=150,
    learning_rate=0.5,
    random_state=42)
```

```
gb.fit(X_train, y_train)
ada.fit(X_train, y_train)

for name, model in [("GradientBoosting", gb), ("AdaBoost", ada)]:
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)
    print(f"{name} - Accuracy: {acc:.3f}")
    print(f"{name} - Confusion matrix:\n{cm}\n")
```

12.5. Usporedimo ih sve skupa



Google colab